RAHUL SHETTY ACADEMY · PLAYWRIGHT INTERVIEW PREP

Playwright Parallel Execution, Workers & Fixture Scope

One interview question, the full 30-minute round · 6 min read

This one question can fill a 30-minute interview round. The short answer is parallel, and that answer leads to many follow-up questions: what parallel actually means, what a worker is, how many you get, how to change the unit of parallelism, and how fixture scope fits in. This deep dive walks through each of them in order.

What does "parallel" actually mean?

Take a suite with 4 test files, 3 tests in each file, 12 tests in total. What does "parallel" mean for this suite?

Option	What it claims
A	All 12 tests run at the same time
B	The 4 files run at the same time, and the 3 tests inside each file run one after another

The answer is B, with one limit: the number of files that really run together is decided by workers.

What is a worker?

A worker is a separate Node.js process that Playwright starts to run your tests. Workers run in parallel with each other, and tests inside one worker run one after another. Workers share no memory, which is why they can run at the same time without interfering.

So the parallelism is between workers, not between tests inside a worker. A worker takes one test, finishes it, then takes the next. With --workers=1, all 12 tests run sequentially in one process.

Example with 4 files, 3 tests each and 2 workers: Playwright keeps a queue of files. Each worker takes one file, runs its 3 tests in order, then asks the queue for the next file.

```
Queue:    file1, file2, file3, file4   (3 tests each)

Worker 0: file1 [t1 → t2 → t3] → file3 [t1 → t2 → t3]   = 6 tests
Worker 1: file2 [t1 → t2 → t3] → file4 [t1 → t2 → t3]   = 6 tests
```

This is one possible outcome, not a fixed plan. Whichever worker becomes free first takes the next file, so slow tests change who gets what.

Workers	What happens to the 12 tests
1	All 12 run one after another
2	2 files at a time, each worker handles 2 files
4	4 files at a time, one file per worker
8	Still only 4 workers start, because there are only 4 files

Does every test get a fresh browser, or a fresh context?

A worker does not open a new browser for every test. It launches one browser and reuses it. Only the context, which is incognito-like, is created fresh for each test.

```
Worker 0 (Node process)
 └── Browser (Chromium)        ← launched ONCE per worker
      ├── Context for test 1   → closed
      ├── Context for test 2   → closed
      └── Context for test 3   → closed
```

Launching a browser takes around a second. Creating a context takes a few milliseconds. Playwright pays the expensive cost once per worker and the cheap cost once per test.

	Job	Cost
Context	Isolation: fresh cookies, localStorage and session, so test 2 cannot be affected by test 1	Very cheap
Worker	Parallelism: a separate process running its own tests at the same time as other workers	Higher: a Node process plus a browser

> In one line: the context answers "will my tests interfere?" and the worker answers "how many run at the same time?" Contexts keep tests independent, and that independence is what makes it safe to spread them across workers.

How many workers do you get if you set nothing?

Playwright uses half of your machine's logical CPU cores, rounded down, with a minimum of 1. The run prints the number at the top, for example "Running 12 tests using 4 workers".

Logical cores	Default workers
4	2
8	4
16	8

```
# check your logical core count, then halve it
node -e "console.log(require('os').cpus().length)"
```

Two things change the number. First, the amount of work: Playwright never starts more workers than it has units of work, so with the default mode and only 2 test files you get at most 2 workers, even on a 16-core machine. Second, your config file. The config generated by npx playwright init contains this line:

```
workers: process.env.CI ? 1 : undefined,
```

undefined means "use the default", so your laptop uses half the cores. On CI, where the CI variable is set, it forces 1 worker. This is a classic interview scenario: tests take 2 minutes locally and 10 minutes on CI, and this line is the cause.

You can set the count yourself in the config, and npx playwright test --workers=6 on the command line overrides the config file:

```
workers: 4,        // fixed number
workers: '50%',    // percentage of logical cores, same as the default
workers: '100%',   // every core
```

> More workers is not always faster. Each worker is a Node process plus a browser, so going past your CPU or RAM makes tests slower and flakier. Half the cores is a safe default because browsers are heavy.

Can tests inside one file also run in parallel?

By default (fullyParallel: false), Playwright hands a whole file to a worker. Test files run in parallel, and the tests inside one file run sequentially in the same worker. With 4 files and 4 workers, each worker takes one file and runs its 3 tests in order:

```
Worker 0: file1 [t1 → t2 → t3]
Worker 1: file2 [t1 → t2 → t3]
Worker 2: file3 [t1 → t2 → t3]
Worker 3: file4 [t1 → t2 → t3]
```

The 3 tests of one file never split across workers. The catch: if you have 1 file with 12 tests, only one worker does the work, because the file is the unit. Useful parallelism is capped at the number of files.

Set fullyParallel: true and the unit of work becomes a single test. All 12 tests go into the queue and each free worker takes the next one, even when its siblings from the same file are running elsewhere.

```
// playwright.config.ts
export default defineConfig({
  workers: 3,
  fullyParallel: true,
});
```

```
Worker 0: file1-t1 → file2-t2 → file4-t1 → ...
Worker 1: file1-t2 → file3-t1 → file2-t3 → ...
Worker 2: file1-t3 → file2-t1 → file3-t2 → ...
```

Again, this is one possible outcome, because assignment is dynamic.

	Default	**fullyParallel: true**
Unit handed to a worker	A whole file	A single test
Tests within one file	Sequential, same worker	Can spread across workers
Most workers that can be busy	Number of files	Number of tests
Hidden risk	Tests may quietly depend on each other's order	That dependency breaks immediately, which helps you find it

What if you want parallel (or serial) for just one file or group?

You do not have to pick one mode for the whole suite. test.describe.configure changes the mode for a single describe block, or for a whole file when you call it at the top level.

```
import { test } from '@playwright/test';

test.describe('Product search', () => {
  test.describe.configure({ mode: 'parallel' }); // tests may spread across workers

  test('search by name', async ({ page }) => { /* ... */ });
  test('search by category', async ({ page }) => { /* ... */ });
});

test.describe('Checkout flow', () => {
  test.describe.configure({ mode: 'serial' });   // run in order, in one worker

  test('add to cart', async ({ page }) => { /* ... */ });
  test('pay', async ({ page }) => { /* ... */ });
  test('verify order', async ({ page }) => { /* ... */ });
});
```

Mode	Behavior
parallel	Tests in the group can run in different workers at the same time, like fullyParallel but for this group only
serial	Tests run in order in one worker. If one fails, the rest of the group is skipped, and a retry re-runs the whole group together

Use serial only for a genuinely dependent flow such as create order, pay, verify. Independent tests are faster and easier to debug, so treat serial as the exception. The two settings combine well: keep fullyParallel: true in the config for speed, and mark only the few dependent groups as serial.

Custom fixtures: why use them over beforeEach/afterEach?

A fixture is setup that Playwright builds for your test, hands to it, and cleans up afterward. It is declared with test.extend and injected by name into the test signature. Code before await use(value) is setup, and code after it is teardown.

Setup is lazy (it runs only for tests that request the fixture), typed and composable (fixtures can depend on other fixtures), and teardown lives next to setup rather than in a distant afterEach. Two extras interviewers like: { auto: true } runs a fixture for every test without it being requested, and mergeTests combines fixture sets from different files.

Test-scoped vs worker-scoped fixtures

Scope decides how often a fixture is built: once per test or once per worker. This is where the browser vs context split pays off.

- **Test scope (default)** — set up before each test and torn down after it. Use it for anything that must be clean every time.
- **Worker scope** — set up once when the worker first needs it, shared by every test that worker runs, torn down when the worker shuts down. Use it for expensive things that are safe to share, such as a DB connection, an API token, a seeded database or a dedicated user account per worker.

Worker scope is declared with the array form [fn, { scope: 'worker' }] and the generic base.extend<TestFixtures, WorkerFixtures>:

```ts
// fixtures.ts
import { test as base } from '@playwright/test';

export const test = base.extend<
  { orderData: string },    // test-scoped fixtures
  { dbConnection: string }  // worker-scoped fixtures
>({
  // Test scope (the default)
  orderData: async ({}, use) => {
    console.log('  TEST fixture setup');
    await use('order-123');           // the test runs here
    console.log('  TEST fixture teardown');
  },

  // Worker scope: array form with the scope option
  dbConnection: [async ({}, use, workerInfo) => {
    console.log(`WORKER ${workerInfo.workerIndex} setup`);
    await use('db-connection');
    console.log(`WORKER ${workerInfo.workerIndex} teardown`);
  }, { scope: 'worker' }],
});

export { expect } from '@playwright/test';
```

```ts
// file1.spec.ts
import { test } from './fixtures';

test('t1', async ({ orderData, dbConnection }) => { console.log('    running t1'); });
test('t2', async ({ orderData, dbConnection }) => { console.log('    running t2'); });
test('t3', async ({ orderData, dbConnection }) => { console.log('    running t3'); });
```

What one worker prints for this file:

```
WORKER 0 setup              ← once
  TEST fixture setup
    running t1
  TEST fixture teardown
  TEST fixture setup
    running t2
  TEST fixture teardown
  TEST fixture setup
    running t3
  TEST fixture teardown
WORKER 0 teardown           ← once, when the worker shuts down
```

Back to the 4 files x 3 tests suite running on 2 workers:

Fixture	Times it runs
Test-scoped (orderData)	12, once per test
Worker-scoped (dbConnection)	2, once per worker

Fixture rules and common traps

- **Fixtures are lazy** — a worker fixture is created only when some test in that worker asks for it. Add { auto: true } in the options to run a fixture for every test even when it is not requested.
- **A failed test restarts the worker** — Playwright discards that worker and starts a new one, and the replacement runs its worker fixtures again. So "once per worker" can happen more times than the worker count.
- **No mutable state in worker scope** — putting something like a shared logged-in page in worker scope brings back coupling between tests.
- **Scope direction** — a worker-scoped fixture can't depend on a test-scoped one.

Interview cheat sheet

Each answer below is short enough to say out loud in an interview.

Question	Crisp answer
Do Playwright tests run in serial or parallel by default?	Parallel. Test files run in parallel across workers, and tests inside one file run sequentially in the same worker.
What is a worker?	A separate Node.js process that runs tests one at a time. Workers run in parallel with each other and share no memory.
Does parallel mean tests or workers run together?	Workers run together. The tests inside a single worker never overlap.
How many workers do you get by default?	Half of the logical CPU cores, and never more than the number of files (or tests, with fullyParallel).
Worker vs browser context?	The context gives each test a clean, isolated state. The worker gives parallelism, and it launches one browser reused by all its tests.
How do you run tests inside one file in parallel?	Set fullyParallel: true in the config, or test.describe.configure({ mode: 'parallel' }) for one group.
How do you force tests to run in order?	test.describe.configure({ mode: 'serial' }). If one test fails, the rest of the group are skipped.
Why are tests slower on CI than locally?	The generated config often sets workers: process.env.CI ? 1 : undefined, which forces a single worker on CI.
What are the fixture scopes?	Test scope (default) runs once per test. Worker scope runs once per worker and is shared by that worker's tests.
Why is browser worker-scoped and page test-scoped?	Launching a browser is expensive, so it is done once per worker. A page must be fresh for each test to keep tests independent.
Are worker fixtures shared across workers?	No. Each worker process creates its own copy.
What happens to the worker when a test fails?	Playwright discards it and starts a new worker, so worker fixtures run again.